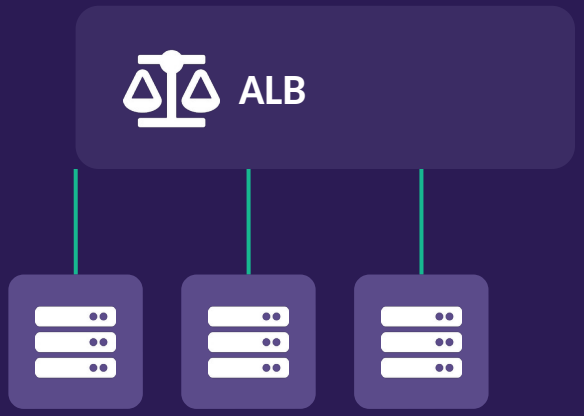
CHAPTER PROJECT 2

AWS Auto Scaling with ALB using Terraform



Building a Self-Healing, Load-Balanced EC2 Fleet with Reusable Modules

Instructor Deck • Terraform Fundamentals Series

OVERVIEW

What We'll Build

VPC with Public & Private Subnets

Reusing our previously built VPC

Application Load Balancer (ALB)

Distributes incoming traffic across instances

Auto Scaling Group (ASG) with EC2 Instances

Automatically scales capacity up or down

Target Group & Health Checks

Ensures traffic only reaches healthy instances

PROJECT LAYOUT

Folder Structure

terraform-autoscaling/

- **main.tf** — *Root config — wires up modules*
- **variables.tf** — *Root-level variables*
- **outputs.tf** — *Root-level outputs*
- **modules/**

alb/

```
main.tf
variables.tf
outputs.tf
```

asg/

```
main.tf
variables.tf
outputs.tf
```

STEP 1

Define the ALB Module

modules/alb/main.tf

```
resource "aws_lb" "my_alb" {
  name               = "my-load-balancer"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [var.alb_sg_id]
  subnets            = var.public_subnets

  tags = {
    Name = "MyALB"
  }
}

resource "aws_lb_target_group" "my_target_group" {
  name     = "my-target-group"
  port     = 80
  protocol = "HTTP"
  vpc_id   = var.vpc_id
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.my_alb.arn
  port              = "80"
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.my_target_group.arn
  }
}
```

Three building blocks

`aws_lb`

The load balancer itself — internet-facing, spans public subnets

`aws_lb_target_group`

A pool of instances the ALB routes HTTP traffic to

`aws_lb_listener`

Listens on port 80 and forwards to the target group

STEP 2

Define the Auto Scaling Group

modules/asg/main.tf

```
resource "aws_launch_template" "my_template" {
  name_prefix   = "asg-template"
  image_id      = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  tag_specifications {
    resource_type = "instance"
    tags = {
      Name = "AutoScalingInstance"
    }
  }
}

resource "aws_autoscaling_group" "my_asg" {
  vpc_zone_identifier = var.private_subnets
  desired_capacity    = 2
  max_size            = 3
  min_size            = 1

  launch_template {
    id      = aws_launch_template.my_template.id
    version = "$Latest"
  }

  target_group_arns = [var.target_group_arn]
}
```

Scaling range

1 **min_size**

2 **desired_capacity**

3 **max_size**

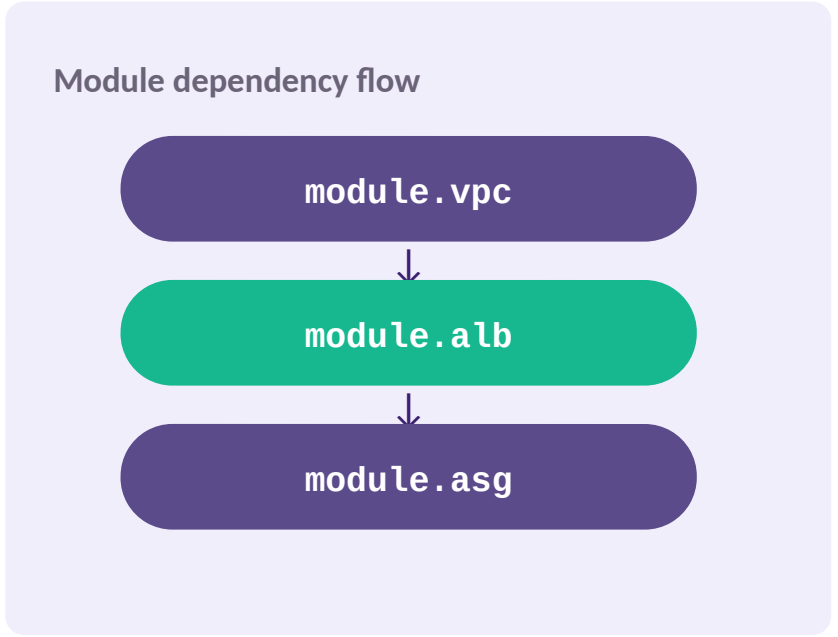
Launch template defines what each instance looks like; the ASG decides how many to run.

STEP 3

Use the Modules in main.tf

```
module "alb" {
  source         = "./modules/alb"
  vpc_id         = module.vpc.vpc_id
  alb_sg_id      = module.security_group.alb_sg_id
  public_subnets = module.vpc.public_subnets
}

module "asg" {
  source           = "./modules/asg"
  private_subnets  = module.vpc.private_subnets
  target_group_arn = module.alb.target_group_arn
}
```



Terraform **resolves this dependency graph automatically** from the references — no manual ordering needed.

STEP 4

Deploy the Auto Scaling Setup

```
$ terraform init
$ terraform apply -auto-approve
```

Request flow

Instances register with the target group automatically as they launch or terminate.

WHAT WE HAVE BUILT

Project 2 Summary

- ALB with a target group for Auto Scaling
- Auto Scaling Group that launches EC2 instances
- Instances automatically register with the ALB

Next: add scaling policies (CPU/target-tracking) and a custom domain via Route 53.